

CSE 226: Fundamentals of Vibe Coding

Mid Exam Study Guide
North South University
Dr. Nabeel Mohammed [NbM]

Spring 2026

Key Concepts

Exam Topics: LLM, Prompting, Context, Agents, API, MCP

Format: 20 MCQs + 1 Broad Question

This guide covers: All 6 topics with **critical facts**, **important concepts**, *examples*, and **common pitfalls**.

Contents

1	LLM (Large Language Models)	3
1.1	Transformer Architecture	3
1.2	How LLMs Work	3
1.3	Key Properties	3
2	Prompting	4
2.1	Prompting Techniques	4
2.2	Prompt Framework: Context + Request + Format	4
2.3	5 Immutable Prompt Principles	4
3	Context	5
3.1	Context Window	5
3.2	The Context Pyramid	5
3.3	Context Management Tools	5
3.4	What to Include in Rules Files	6
4	Agents	6
4.1	How Agents Work	6
4.2	Key Agent Capabilities	6
4.3	Agent Tools	7
4.4	When to Use Agents	7
5	API (Application Programming Interface)	7
5.1	LLM API Components	7
5.2	Key API Concepts	7
5.3	API Request Structure (Example)	8

6	MCP (Model Context Protocol)	8
6.1	The N×M Problem	8
6.2	MCP Architecture	8
6.3	Three Core Primitives	9
6.4	Key Technical Details	9
6.5	Adoption (as of 2026)	9
6.6	MCP Security Concerns	9
6.7	MCP vs Traditional API	10
7	40 Practice MCQs with Answers	11

1 LLM (Large Language Models)

Definition

Large Language Models (LLMs) are AI models trained on massive text datasets to understand and generate human-like text. They are based on the **Transformer architecture** introduced in the 2017 paper *"Attention Is All You Need"* by Vaswani et al.

1.1 Transformer Architecture

- **Self-Attention**: Mechanism that allows the model to weigh the importance of different words in a sequence simultaneously.
- **Encoder-Decoder**: Original Transformer has both; GPT uses **decoder-only** architecture.
- **Parallel Processing**: Unlike RNNs, Transformers process all tokens simultaneously, enabling massive scale.
- **Positional Encoding**: Adds position information since Transformers have no inherent notion of word order.

1.2 How LLMs Work

- **Tokens**: Text is broken into tokens (words, subwords, or characters). *"unhappiness" → "un" + "happiness"*.
- **Pre-training**: Learns language patterns from massive datasets via **next-token prediction** (unsupervised learning).
- **Fine-tuning**: Specialized training on specific tasks or domains.
- **RLHF** (Reinforcement Learning from Human Feedback): Aligns model outputs with human preferences.
- **Parameters**: Weights learned during training. *GPT-4 has ~1.76 trillion parameters*.
- **Context Window**: Maximum input length the model can process. *Claude supports 200K+ tokens*.

1.3 Key Properties

- **Probabilistic**: LLMs predict the next token based on probability distributions, **NOT deterministic**.
- **Temperature**: Controls output randomness. *High (0.8-1.0) = creative; Low (0.1-0.3) = deterministic*.
- **Hallucination**: When LLMs generate plausible but factually incorrect information.
- **Top-p (Nucleus Sampling)**: Limits token selection to top cumulative probability mass.

Examples

Major LLMs: GPT-4/4o (OpenAI), Claude 3.5/3.7 Sonnet (Anthropic), Gemini (Google DeepMind), Llama (Meta)

Important Warnings

LLMs do **NOT understand** text the way humans do. They are sophisticated pattern-matching engines based on statistical prediction.

2 Prompting

Definition

Prompt Engineering is the process of designing and optimizing text inputs (prompts) to deliver consistent, high-quality responses from LLMs.

2.1 Prompting Techniques

1. **Zero-Shot Prompting**: No examples given; just the instruction.
 - *Prompt: "Translate this to French: Hello, how are you?"*
2. **One-Shot Prompting**: One example provided before the task.
 - *Prompt: "English: Good morning → French: Bonjour. English: Good night → French: ?"*
3. **Few-Shot Prompting**: Multiple examples provided to guide the model.
 - *Provide 3-5 examples before asking the actual question.*
 - **Few-shot consistently outperforms zero-shot for complex tasks.**
4. **Chain-of-Thought (CoT)**: Forces the model to show step-by-step reasoning.
 - *Add "Think step by step" to the prompt.*
 - **Significantly improves performance on math, logic, and reasoning tasks.**
5. **Tree-of-Thought (ToT)**: Explores multiple reasoning paths simultaneously, then selects the best.
6. **ReAct (Reasoning + Acting)**: Model alternates between thinking (reasoning traces) and using tools (actions).

2.2 Prompt Framework: Context + Request + Format

- **Context**: What the AI needs to know about the codebase or problem space.
- **Request**: The specific implementation or answer wanted.
- **Format**: How the AI should respond (e.g., "explain first, then provide code").

2.3 5 Immutable Prompt Principles

1. **Assume Zero Tribal Knowledge**: AI doesn't know your coding standards unless stated.
2. **Constrain the Output**: Specify exactly what you want (e.g., "Output ONLY the function").
3. **Use Negative Prompts**: Tell the AI what **NOT** to do.

4. **Chain of Thought Formatting**: For complex tasks, ask the model to break down logic first.
5. **Iterate in Context**: Refine within the same conversation; don't restart from scratch.

Important Warnings

Weak prompt: "Make a login page."

Strong prompt: "Build a React (Next.js App Router) Login component using TypeScript with strict typing, client-side validation for email and 8-char minimum password, CSS modules styling, and three API error states."

3 Context

Definition

Context Engineering is the practice of structuring the environment (documentation, schemas, constraints, rules files) so the AI produces correct results consistently. It is considered the **successor to prompt engineering**.

3.1 Context Window

- **Definition**: The maximum amount of text/tokens an LLM can process in a single interaction.
- **"Lost in the Middle" Syndrome**: Models perform worse on information placed in the middle of long contexts. **Put critical info at the beginning or end.**
- *Claude: 200K+ tokens; GPT-4: 128K tokens.*

3.2 The Context Pyramid

1. **Core (Always Include)**: Current file, exact error log, direct type definitions/dependencies.
2. **Periphery (As Needed)**: Related database schemas, global design tokens, utility functions.
3. **Rules (Summarize Only)**: Architecture mandates, authentication protocols, project documentation.

3.3 Context Management Tools

- **Rules Files**: Persistent, reusable context included at the start of every AI interaction.
 - *Cursor*: `.cursor/rules/` directory with `.mdc` files
 - *Claude Code*: `CLAUDE.md` in project root
 - *Windsurf*: `.windsurfrules`
- **@ Targeting**: Explicitly reference specific files in the context window (e.g., `@file.ts @docs.md`).
- **Context Resetting**: Start a fresh chat when solving a new sub-problem. Long threads cause confusion from past failed experiments.

Important Warnings

Critical Error: Highlighting the entire workspace and saying "fix the app." Supplying **too much context** degrades model performance.

3.4 What to Include in Rules Files

- **Always:** Tech stack and versions, file organization conventions, naming patterns, error handling approach, testing expectations.
- **Context-dependent:** Current sprint/focus area, known issues, integration patterns.
- **Don't include:** Obvious things ("write clean code"), frequently changing information, irrelevant personal preferences.

4 Agents

Definition

AI Agents are AI systems that execute multi-step tasks autonomously. Unlike basic LLM chat (prompt → response), agents can **plan, execute, self-correct, and use tools** without human intervention at each step.

4.1 How Agents Work

1. Receive a **goal** (not just a prompt)
2. **Break down** the goal into sub-tasks
3. **Execute** each step autonomously
4. **Handle errors** and adjust course
5. Present **final result** for human review

Examples

Traditional Vibe Coding: You prompt → AI generates code → You review → You prompt again

Agent-Based Vibe Coding: You describe goal → Agent plans → Agent executes all steps → You review once

4.2 Key Agent Capabilities

- **Tool Calling / Function Calling:** LLMs invoke external functions, APIs, and tools.
- **Multi-file Edits:** Modify many files simultaneously in one operation.
- **Code Execution:** Run shell commands, execute tests, run scripts.
- **Self-Correction:** Detect errors in output and fix them autonomously.
- **Dynamic Discovery:** Discover available tools at runtime.

4.3 Agent Tools

- **Cursor Agent Mode:** Multi-file autonomous changes with Composer feature.
- **Claude Code:** Terminal agent that reads/writes files, runs shell commands, executes tests.
- **Windsurf Cascade:** Deep agentic workflows with autonomous context pulling.

4.4 When to Use Agents

- **Good for:** Refactoring across many files, implementing multi-layer features, migrations, test generation.
- **Less good for:** Exploratory coding, security-sensitive changes, situations where intermediate states matter.

Important Warnings

Guardrails needed: Permission boundaries, human-in-the-loop for critical actions, audit logging, input validation. **Never give agents production database write access without safeguards.**

5 API (Application Programming Interface)

Definition

API is a set of rules and protocols that allow different software applications to communicate with each other. In the context of LLMs, APIs provide programmatic access to AI models.

5.1 LLM API Components

- **Endpoint:** The URL where the API is accessed. <https://api.openai.com/v1/chat/completions>
- **Authentication:** API keys for access control. **Never expose API keys in client-side code.**
- **Request:** Contains the model name, prompt/messages, and parameters (temperature, max_tokens, etc.).
- **Response:** Contains the generated text, token usage statistics, and metadata.

5.2 Key API Concepts

- **Rate Limiting:** Controls how many requests can be made per time period (e.g., requests per minute).
- **Token Usage:** APIs charge per token (both input and output tokens count).
- **Streaming:** Receive response tokens as they are generated instead of waiting for the complete response.
- **Function Calling via API:** Define tools/schemas in the API request; the LLM decides when and which to call.

- **REST API**: Most common architecture for LLM services (uses HTTP methods: GET, POST, etc.).

Examples

Major LLM API Providers: OpenAI API, Anthropic Claude API, Google Gemini API, Together AI, Groq

5.3 API Request Structure (Example)

- **Model**: Which model to use (e.g., `gpt-4o`, `claude-3-5-sonnet`)
- **Messages**: Array of role-based messages (system, user, assistant)
- **Parameters**: temperature, max_tokens, top_p, stop sequences
- **Tools**: Optional function/tool definitions for function calling

6 MCP (Model Context Protocol)

Definition

Model Context Protocol (MCP) is an **open standard** introduced by **Anthropic** in **November 2024** that standardizes how AI models connect to external tools, data sources, and systems. In **December 2025**, Anthropic donated MCP to the **Agentic AI Foundation** under the **Linux Foundation**, making it vendor-neutral.

6.1 The N×M Problem

- **Before MCP**: $N \text{ models} \times M \text{ tools} = N \times M$ custom integrations needed.
- **After MCP**: $N + M$ integrations (each model implements MCP once; each tool implements MCP once).
- **Analogy**: MCP is the **USB-C** for AI – a universal connector.

6.2 MCP Architecture

- **MCP Host**: The application that wants to use AI capabilities. *Claude Desktop, VS Code, Cursor, custom applications, LangChain.*
- **MCP Client**: Component within the host that maintains connections to MCP servers.
- **MCP Server**: Service that exposes tools, data, and prompts through the standardized MCP interface.

Examples

Flow: [Your Application] → [MCP Client] → [MCP Server] → [External Tool/Data]

6.3 Three Core Primitives

1. **Tools:** Functions the model can call. Have defined input schemas and output formats.
 - *create_issue, query_database, search_codebase*
2. **Resources:** Data the model can read. Have URIs and can be static or dynamic.
 - *Files, database records, API responses, structured content.*
3. **Prompts:** Reusable templates that guide how the model interacts with the server's capabilities.
 - *"triage incoming ticket", "draft response to billing question"*

6.4 Key Technical Details

- **Transport Protocol: JSON-RPC 2.0** (same message format as the Language Server Protocol).
- **Dynamic Discovery:** Tools are discovered at runtime. Server responds with available tools, resources, and prompts including their schemas.
- **Server-side updates:** Add or update tools on the server without changing anything on the client side.

6.5 Adoption (as of 2026)

- **AI Platforms:** Anthropic (Claude Desktop, Claude Code), OpenAI (adopted March 2025), Google DeepMind (Gemini), Microsoft (Semantic Kernel, Azure OpenAI).
- **Development Tools:** Cursor, Windsurf, VS Code (Copilot), Replit, Sourcegraph.
- **Automation:** n8n, Zapier, Playwright.
- **Enterprise:** Supabase, Cloudflare, Amazon Bedrock AgentCore.
- **SDKs available:** Python, TypeScript, C#, Java.

6.6 MCP Security Concerns

- **Prompt Injection through Tool Results:** Malicious data returned by an MCP server could contain injected instructions.
- **Over-Permissioned Tools:** Poorly scoped permissions can allow agents to perform unauthorized actions.
- **Tool Impersonation/Poisoning:** Malicious MCP servers advertise tools with names similar to trusted ones.
- **No Default Authentication:** Authentication between clients and servers is left to implementors.

Important Warnings

Best Practices: Scope tool permissions narrowly, require user approval for write operations, validate all inputs/outputs, use authenticated MCP connections (TLS + API keys), monitor and audit all invocations, never expose MCP servers to the public internet without authentication.

6.7 MCP vs Traditional API

Aspect	Traditional APIs	MCP
Discovery	Manual documentation	Automatic at runtime
Schema	Per-API format	Standardized JSON schema
Updates	Requires client changes	Server-side only
Multi-model	One integration per model	Write once, works everywhere

Key Concepts

MCP does NOT replace APIs. It sits on top of them. An MCP server wraps existing APIs in the standardized MCP interface.

7 40 Practice MCQs with Answers

LLM (Questions 1–7)

Question 1

What architecture are all modern Large Language Models based on?

- A) Recurrent Neural Network (RNN)
- B) Convolutional Neural Network (CNN)
- C) **Transformer**
- D) Generative Adversarial Network (GAN)

Answer: C) Transformer – Introduced in the 2017 paper "Attention Is All You Need" by Vaswani et al.

Question 2

What does the self-attention mechanism in Transformers allow the model to do?

- A) Process images
- B) **Weigh the importance of different words in a sequence simultaneously**
- C) Reduce the number of parameters
- D) Convert text to speech

Answer: B) Self-attention allows the model to consider all tokens in a sequence at once and determine their relative importance.

Question 3

What does RLHF stand for?

- A) Random Learning with Human Feedback
- B) **Reinforcement Learning from Human Feedback**
- C) Regulated Language Human Framework
- D) Recursive Learning for Human Functions

Answer: B) RLHF aligns model outputs with human preferences through reinforcement learning.

Question 4

What does the temperature parameter control in an LLM?

- A) Processing speed
- B) Model size
- C) **Randomness/creativity of the output**
- D) Number of parameters

Answer: C) High temperature (0.8-1.0) = more creative/random; Low temperature (0.1-0.3) = more deterministic.

Question 5

What is "hallucination" in the context of LLMs?

- A) The model refusing to answer
- B) **Generating plausible but factually incorrect information**
- C) The model generating code only
- D) The model crashing during inference

Answer: B) Hallucination occurs when LLMs produce confident-sounding but incorrect or fabricated information.

Question 6

LLMs are fundamentally _____ systems.

- A) Deterministic
- B) Rule-based
- C) **Probabilistic**
- D) Symbolic

Answer: C) LLMs predict the next token based on probability distributions, not fixed rules.

Question 7

What is a "token" in the context of LLMs?

- A) A security credential
- B) **A unit of text (word, subword, or character) that the model processes**
- C) A type of neural network layer
- D) A training dataset

Answer: B) Text is broken into tokens before processing. *"unhappiness" → "un" + "happiness"*.

Prompting (Questions 8–14)**Question 8**

Which prompting technique provides NO examples to the model?

- A) Few-shot
- B) One-shot
- C) **Zero-shot**
- D) Chain-of-Thought

Answer: C) Zero-shot prompting gives only the instruction without any examples.

Question 9

Which prompting technique is MOST effective for complex reasoning tasks?

- A) Zero-shot
- B) **Chain-of-Thought (CoT)**
- C) One-shot
- D) Direct instruction

Answer: B) Chain-of-Thought ("think step by step") significantly improves performance on math, logic, and reasoning tasks.

Question 10

What does ReAct stand for in prompting?

- A) React and Act
- B) **Reasoning + Acting**
- C) Response and Action
- D) Rapid Extraction

Answer: B) ReAct combines reasoning traces with tool actions, alternating between thinking and doing.

Question 11

What is the recommended prompt framework structure?

- A) Input → Process → Output
- B) **Context + Request + Format**
- C) Question → Answer → Verify
- D) Problem → Solution → Test

Answer: B) Context (what AI needs to know) + Request (what you want) + Format (how to respond).

Question 12

What is a "negative prompt"?

- A) A prompt that produces errors
- B) **Telling the AI what NOT to do**
- C) A prompt with no context
- D) A prompt that decreases temperature

Answer: B) Negative prompts explicitly constrain the AI's behavior. *"Do not use React Context, use local state."*

Question 13

Why is few-shot prompting generally better than zero-shot for complex tasks?

- A) It uses less computation
- B) It requires fewer tokens
- C) **Examples provide concrete patterns that guide the model's output**
- D) Zero-shot is not a real technique

Answer: C) Examples demonstrate the expected format, style, and reasoning pattern.

Question 14

What is Tree-of-Thought (ToT) prompting?

- A) A technique for generating tree data structures
- B) **Exploring multiple reasoning paths simultaneously and selecting the best**
- C) A method for organizing prompts hierarchically
- D) A way to visualize model attention

Answer: B) ToT extends Chain-of-Thought by exploring multiple reasoning branches.

Context (Questions 15–20)**Question 15**

What is Context Engineering?

- A) Building context windows from scratch
- B) **Structuring the environment so the AI produces correct results consistently**
- C) Writing better prompts
- D) Compressing text to fit in context windows

Answer: B) Context engineering is considered the successor to prompt engineering.

Question 16

What is the "Lost in the Middle" syndrome?

- A) The model loses network connection
- B) **Models perform worse on information placed in the middle of long contexts**
- C) The model forgets the beginning of the prompt
- D) The context window gets corrupted

Answer: B) Critical information should be placed at the beginning or end of the context, not the middle.

Question 17

Which file is used for persistent context in Claude Code?

- A) `.cursorrules`
- B) **`CLAUDE.md`**
- C) `.windsurfrules`
- D) `context.txt`

Answer: B) Claude Code uses `CLAUDE.md` in the project root for persistent context.

Question 18

What is "Context Resetting" and why is it important?

- A) Clearing the model's weights; it improves accuracy
- B) **Starting a fresh chat for new sub-problems; long threads cause confusion from past failed experiments**
- C) Deleting the project files; it removes bugs
- D) Restarting the IDE; it fixes memory leaks

Answer: B) Reusing the same chat thread for too long causes the AI to remember bad code and deprecated ideas.

Question 19

What should you NOT include in rules files?

- A) Tech stack and versions
- B) File organization conventions
- C) **Obvious things like "write clean code" and frequently changing information**
- D) Error handling approach

Answer: C) Rules files should contain specific, actionable conventions, not vague instructions.

Question 20

What is the biggest mistake beginners make with context?

- A) Using too few tokens
- B) **Supplying too much context (e.g., entire workspace), which degrades performance**
- C) Not using temperature settings
- D) Using the wrong model

Answer: B) Overloading the context window with irrelevant information reduces model performance.

Agents (Questions 21–27)**Question 21**

What fundamentally distinguishes AI Agents from basic LLM chat?

- A) Agents are faster
- B) Agents use more tokens
- C) **Agents execute multi-step tasks autonomously**
- D) Agents only work with code

Answer: C) Agents can plan, execute, self-correct, and use tools without human intervention at each step.

Question 22

What is "Tool Calling" (also called "Function Calling")?

- A) The user calling the AI on the phone
- B) **The mechanism that enables LLMs to invoke external functions, APIs, and tools**
- C) Calling a function in Python
- D) A debugging technique

Answer: B) Tool calling transforms LLMs from passive text generators into action-taking agents.

Question 23

Which of the following is a capability of AI agents?

- A) Only generating text
- B) **Multi-file edits, code execution, self-correction, and tool use**
- C) Only answering questions
- D) Only writing documentation

Answer: B) Agents can modify multiple files, run commands, execute tests, and correct their own errors.

Question 24

When should you AVOID using AI agents?

- A) For refactoring across many files
- B) **For security-sensitive changes that need human review at each step**
- C) For test generation
- D) For implementing features that touch multiple layers

Answer: B) Security-sensitive changes always require human review at each step.

Question 25

What is "Dynamic Discovery" in the context of AI agents?

- A) The agent discovers new programming languages
- B) **The agent discovers available tools at runtime without pre-configuration**
- C) The agent dynamically changes its temperature
- D) The agent discovers bugs in the code

Answer: B) Agents can query and discover what tools are available at runtime.

Question 26

Which tool is described as a "terminal agent" that can read/write files and run shell commands?

- A) Cursor Composer
- B) **Claude Code**
- C) GitHub Copilot
- D) v0.dev

Answer: B) Claude Code operates as a terminal agent by default.

Question 27

What is the recommended order of operations for an AI agent workflow?

- A) Execute → Plan → Review
- B) Review → Execute → Plan
- C) **Plan → Execute → Review**
- D) Review → Plan → Execute

Answer: C) The agent plans the steps, executes them autonomously, then presents results for review.

API (Questions 28–33)**Question 28**

What does API stand for?

- A) Artificial Programming Interface
- B) **Application Programming Interface**
- C) Automated Process Integration
- D) Advanced Protocol Interface

Answer: B) API defines how software components communicate with each other.

Question 29

What is "streaming" in the context of LLM APIs?

- A) Streaming video from the model
- B) **Receiving response tokens as they are generated instead of waiting for the complete response**
- C) Streaming audio to the model
- D) Sending multiple requests simultaneously

Answer: B) Streaming provides a better user experience by showing output progressively.

Question 30

How do LLM APIs typically charge for usage?

- A) Per request only
- B) **Per token (both input and output tokens)**
- C) Per hour of usage
- D) Flat monthly fee only

Answer: B) Token usage is the primary pricing metric for LLM APIs.

Question 31

What is "rate limiting" in API usage?

- A) Limiting the model's creativity
- B) **Controlling how many requests can be made per time period**
- C) Limiting the response length
- D) Reducing the model's temperature

Answer: B) Rate limiting prevents abuse and ensures fair usage (e.g., requests per minute).

Question 32

What is "function calling" via API?

- A) Calling a phone number through the API
- B) **Defining tools/schemas in the API request; the LLM decides when and which to call**
- C) Calling the API from a function
- D) A type of error handling

Answer: B) Function calling allows the LLM to structure its output to invoke external tools.

Question 33

Which architecture is most commonly used for LLM APIs?

- A) GraphQL
- B) gRPC
- C) **REST API**
- D) WebSocket only

Answer: C) REST API (using HTTP methods like POST) is the most common architecture for LLM services.

MCP (Questions 34–40)**Question 34**

Who introduced the Model Context Protocol (MCP) and when?

- A) OpenAI in January 2025
- B) **Anthropic in November 2024**
- C) Google in March 2024
- D) Microsoft in December 2023

Answer: B) Anthropic introduced MCP in November 2024. It was later donated to the Linux Foundation in December 2025.

Question 35

What problem does MCP solve?

- A) The halting problem
- B) **The $N \times M$ integration problem (custom connectors for every model-tool combination)**
- C) The traveling salesman problem
- D) Network latency problems

Answer: B) MCP converts $N \times M$ integrations into $N + M$ by creating a universal standard.

Question 36

What are the three core primitives of MCP?

- A) Input, Output, Process
- B) Request, Response, Error
- C) **Tools, Resources, Prompts**
- D) Client, Server, Protocol

Answer: C) Tools (functions to call), Resources (data to read), and Prompts (reusable templates).

Question 37

What transport protocol does MCP use?

- A) HTTP/2
- B) WebSocket
- C) gRPC
- D) **JSON-RPC 2.0**

Answer: D) MCP uses JSON-RPC 2.0, the same message format as the Language Server Protocol.

Question 38

What is "Dynamic Discovery" in MCP?

- A) The server dynamically changes its IP address
- B) **Tools are discovered at runtime; the server responds with available capabilities without client changes**
- C) The model dynamically adjusts its parameters
- D) The client dynamically loads plugins

Answer: B) When a client connects, it queries the server's capabilities at runtime.

Question 39

Which of the following is a major MCP security concern?

- A) Slow response times
- B) **Prompt injection through tool results and over-permissioned tools**
- C) High token usage
- D) Model hallucination

Answer: B) Malicious data in tool results could inject instructions, and broad permissions could allow unauthorized actions.

Question 40

Does MCP replace traditional APIs?

- A) Yes, MCP makes APIs obsolete
- B) **No, MCP sits on top of APIs and wraps them in a standardized interface**
- C) Yes, but only for REST APIs
- D) No, MCP is unrelated to APIs

Answer: B) An MCP server wraps existing APIs in the standardized MCP interface, making them accessible to any MCP-compatible model.

Answer Key Summary

Q#	Answer	Q#	Answer	Q#	Answer
1	C	15	B	29	B
2	B	16	B	30	B
3	B	17	B	31	B
4	C	18	B	32	B
5	B	19	C	33	C
6	C	20	B	34	B
7	B	21	C	35	B
8	C	22	B	36	C
9	B	23	B	37	D
10	B	24	B	38	B
11	B	25	B	39	B
12	B	26	B	40	B
13	C	27	C		
14	B	28	B		

Key Concepts

Color Legend:

Red = Critical definitions / must-know facts

Blue = Important concepts

Green = Examples

Orange = Warnings / common pitfalls

Purple = MCQ answers and explanations